LABSHEET 5

A. INNER JOIN

1. Demonstrate an INNER JOIN between Student and Department.

```
SELECT s.Student_ID, s.Student_Name, d.Department_Name
FROM Student AS s
INNER JOIN Department AS d
 ON s.Department_ID = d.Department_ID;
```

2. Display student names along with their department names using INNER JOIN.

```
SELECT s.Student_Name, d.Department_Name
FROM Student AS s
INNER JOIN Department AS d
 ON s.Department_ID = d.Department_ID;
```

3. Display employee names along with their department names.

```
SELECT e.Emp_Name, d.Department_Name
FROM Employee AS e
INNER JOIN Department AS d
 ON e.Department_ID = d.Department_ID;
```

4. Display students along with their course details.

```
SELECT s.Student_ID, s.Student_Name,
 c.Course_ID, c.Course_Name
FROM Student AS s
INNER JOIN Course AS c
 ON s.Course_ID = c.Course_ID;
```

5. Find students belonging to a particular department using INNER JOIN.

```
SELECT s.Student_ID, s.Student_Name, d.Department_Name
FROM Student AS s
INNER JOIN Department AS d
 ON s.Department_ID = d.Department_ID
WHERE d.Department_Name = 'Computer Science';
```

B. LEFT and RIGHT OUTER JOIN

6. Demonstrate a LEFT OUTER JOIN between Student and Department.

```
SELECT s.Student_ID, s.Student_Name, d.Department_Name
FROM Student AS s
LEFT OUTER JOIN Department AS d
 ON s.Department_ID = d.Department_ID;
```

7. Display all students, including those who do not have a matching department.

```
SELECT s.Student_ID, s.Student_Name,
d.Department_Name
FROM Student AS s
LEFT JOIN Department AS d
 ON s.Department_ID = d.Department_ID;
```

8. Demonstrate a RIGHT OUTER JOIN between Student and Department.

```
SELECT s.Student_ID, s.Student_Name, d.Department_ID,
d.Department_Name
FROM Student AS s
RIGHT OUTER JOIN Department AS d
 ON s.Department_ID = d.Department_ID;
```

9. Display all departments, including departments having no students.

```
SELECT d.Department_ID, d.Department_Name,
 s.Student_ID, s.Student_Name
FROM Department AS d
LEFT JOIN Student AS s
 ON d.Department_ID = s.Department_ID;
```

10. Find students who are not assigned to any department using LEFT JOIN.

```
SELECT s.Student_ID, s.Student_Name
FROM Student AS s
LEFT JOIN Department AS d
 ON s.Department_ID = d.Department_ID
WHERE d.Department_ID IS NULL;
```

C. Unmatched Rows and Other Joins

11. Find departments having no students using RIGHT/LEFT JOIN.

```
SELECT d.Department_ID, d.Department_Name
FROM Department AS d
LEFT JOIN Student AS s
 ON d.Department_ID = s.Department_ID
WHERE s.Student_ID IS NULL;
```

12. Demonstrate FULL OUTER JOIN concept using a MySQL workaround.

```
SELECT s.Student_ID, s.Student_Name, d.Department_ID,
d.Department_Name
FROM Student AS s
LEFT JOIN Department AS d
 ON s.Department_ID = d.Department_ID
UNION
SELECT s.Student_ID, s.Student_Name, d.Department_ID,
d.Department_Name
FROM Student AS s
RIGHT JOIN Department AS d
 ON s.Department_ID = d.Department_ID;
```

13. Demonstrate a CROSS JOIN between Student and Course.

```
SELECT s.Student_Name, c.Course_Name
FROM Student AS s
CROSS JOIN Course AS c;
```

14. Calculate the number of possible Student-Course combinations using CROSS JOIN.

```
SELECT COUNT(*) AS Total_Combinations
FROM Student AS s
CROSS JOIN Course AS c;
```

15. Demonstrate a NATURAL JOIN between two suitable tables.

```
SELECT Student_ID, Student_Name, Department_Name
FROM Student
NATURAL JOIN Department;
```

16. Compare the result of NATURAL JOIN and INNER JOIN.

```
FROM Student NATURAL JOIN Department;
SELECT s.Student_ID, s.Student_Name, d.Department_Name
FROM Student AS s
INNER JOIN Department AS d
 ON s.Department_ID = d.Department_ID;
```

D. SELF JOIN and Multiple Joins

17. Perform a SELF JOIN on Employee to show employee-manager relationships.

```
SELECT e.Emp_Name AS Employee_Name,
  m.Emp_Name AS Manager_Name
 FROM Employee AS e
 LEFT JOIN Employee AS m
  ON e.Manager_ID = m.Emp_ID;
```

18. Display employees along with their managers using SELF JOIN.

```
 SELECT e.Emp_ID, e.Emp_Name,
  m.Emp_Name AS Manager_Name
 FROM Employee AS e
 LEFT JOIN Employee AS m
  ON e.Manager_ID = m.Emp_ID;
```

19. Join three tables—Student, Department and Course.

```
 SELECT s.Student_Name, d.Department_Name, c.Course_Name
 FROM Student AS s
 INNER JOIN Department AS d
  ON s.Department_ID = d.Department_ID
 INNER JOIN Course AS c
  ON s.Course_ID = c.Course_ID;
```

20. Display student name, course name and department name using multiple joins.

```
SELECT s.Student_Name, c.Course_Name,
d.Department_Name
FROM Student AS s
LEFT JOIN Course AS c
 ON s.Course_ID = c.Course_ID
LEFT JOIN Department AS d
 ON s.Department_ID = d.Department_ID;
```

E. Joins with Aggregation and Subqueries

21. Find employees whose salary is greater than the average salary of their department.

```
SELECT e.Emp_ID, e.Emp_Name, e.Department_ID,
e.Salary
FROM Employee AS e
WHERE e.Salary > (
 SELECT AVG(e2.Salary)
 FROM Employee AS e2
 WHERE e2.Department_ID = e.Department_ID
);
```

22. Find the department having the maximum number of students using joins and aggregation.

```
SELECT d.Department_ID, d.Department_Name,
  COUNT(s.Student_ID) AS Student_Count
FROM Department AS d
LEFT JOIN Student AS s
 ON d.Department_ID = s.Department_ID
GROUP BY d.Department_ID, d.Department_Name
HAVING COUNT(s.Student_ID) = (
 SELECT MAX(dept_count)
 FROM (
 SELECT COUNT(s2.Student_ID) AS dept_count
 FROM Department AS d2
 LEFT JOIN Student AS s2
 ON d2.Department_ID = s2.Department_ID
 GROUP BY d2.Department_ID
 ) AS counts_by_department
);
```

23. Display departments and the total number of students in each department.

```
SELECT d.Department_ID, d.Department_Name,
 COUNT(s.Student_ID) AS Total_Students
FROM Department AS d
LEFT JOIN Student AS s
 ON d.Department_ID = s.Department_ID
GROUP BY d.Department_ID, d.Department_Name
ORDER BY d.Department_Name;
```

24. Display courses along with the number of students enrolled in each course.

```
SELECT c.Course_ID, c.Course_Name,
 COUNT(s.Student_ID) AS Enrolled_Students
FROM Course AS c
LEFT JOIN Student AS s
 ON c.Course_ID = s.Course_ID
```

```
 GROUP BY c.Course_ID, c.Course_Name
 ORDER BY c.Course_Name;
```

25. Create a report using INNER JOIN, LEFT JOIN, GROUP BY and aggregate functions. SELECT

```
d.Department_ID, d.Department_Name,
  c.Course_ID, c.Course_Name,
  COUNT(s.Student_ID) AS Student_Count
 FROM Department AS d
 INNER JOIN Course AS c
  ON c.Department_ID = d.Department_ID
 LEFT JOIN Student AS s
  ON s.Department_ID = d.Department_ID
  AND s.Course_ID = c.Course_ID
 GROUP BY d.Department_ID, d.Department_Name,
  c.Course_ID, c.Course_Name
 ORDER BY d.Department_Name, c.Course_Name;
```